Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

Tutorial 5 : Reconnaître le matériau avec l'Arduino Portenta H7 : modèle TFLite INT8, mémoire et latence dans OpenMV IDE

Séries de tuto sur Portenta H7, lecture 5 « Reconnaître le matériau »

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

2 novembre 2026

Objectifs du Tutorial

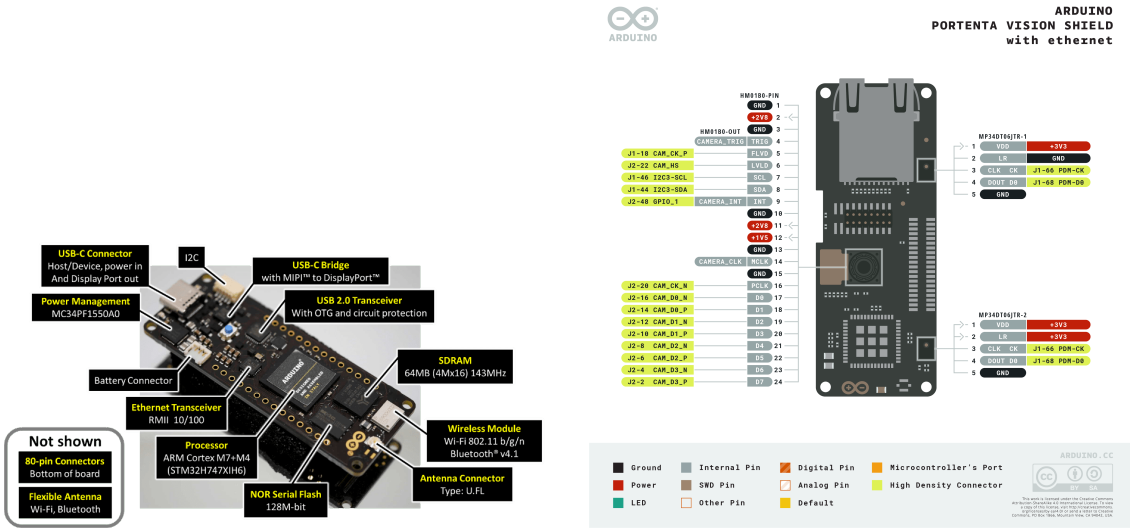
Dans ce tutoriel, la Portenta H7 devient l'unité de décision de la station EcoTri 4.0 : elle charge un réseau de neurones quantifié en INT8, classe en temps réel l'objet sous la caméra (carton, métal, plastique, verre) et mesure ce que ce choix lui coûte. Chaque mesure est reliée au cours : paramètres et Flash, pic d'activations et RAM, MACs et latence, quantification et exactitude. À la fin de ce tutoriel, vous serez capable de :

- Produire, sur le PC, un modèle `.tflite` float32 et sa version INT8 quantifiée après entraînement.
- Inspecter un modèle TFLite (forme et type des tenseurs, paramètres de quantification, taille).
- Charger le modèle avec le module `ml` d'OpenMV et mesurer la mémoire consommée.
- Classer le flux de la caméra, mesurer la latence d'inférence et les images par seconde.
- Vérifier la cohérence entre les prédictions du PC et celles de la carte.
- Ajouter un seuil de confiance et une décision de tri publiée en JSON.

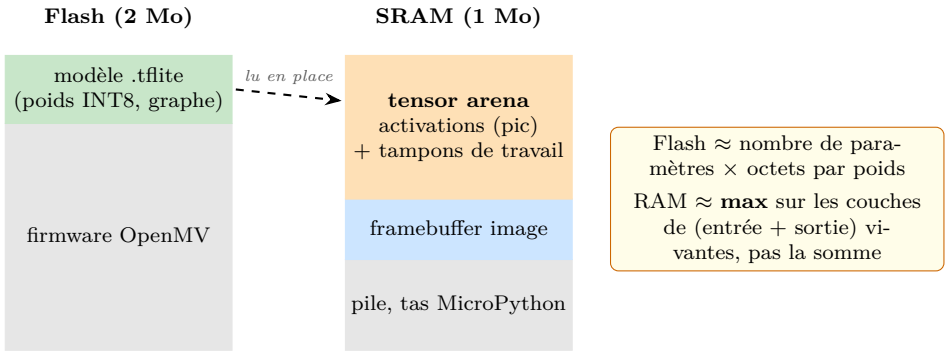
Durée indicative : 1 h 15. Ce tutoriel prépare le TP5 (comparaison de trois modèles, front de Pareto et jalon J5).

1 Introduction : un réseau de neurones dans 1 Mo de RAM

Le firmware OpenMV embarque **TensorFlow Lite Micro** et les noyaux **CMSIS-NN**. Un modèle `.tflite` copié sur la carte est lu par le module `ml` : ses poids restent dans un tampon en mémoire, ses activations sont placées dans une zone de travail (*tensor arena*). Le Cortex-M7 à 480 MHz exécute ensuite les convolutions INT8 avec ses instructions SIMD.



La Portenta H7 et son Vision Shield : capteur HM01B0 (320×240, niveaux de gris) ou caméra couleur selon la version ; le modèle de ce tutoriel accepte les deux.



Rappel du cours : les poids occupent la Flash (ou le tas, une fois chargés), les activations occupent l'arena ; le besoin en RAM est le pic sur les couches.

Ce que nous allons construire

Étape	Programme	Notion du cours
1	`entrainer_convertir.py` (PC)	transfer learning, PTQ INT8, calibration
2	`inspecter_tflite.py` (PC)	tenseurs, échelle s, point zéro z
3	`ml_charger.py`	Flash, RAM, `load_to_fb`
4–5	`ml_classer.py`	latence, FPS, prétraitement
6	`ml_coherence.py`	écart PC / carte
7	`ml_decision.py`	seuil de confiance, décision JSON

2 Pré-requis

Matériel

- Carte **Arduino Portenta H7** avec son **Vision Shield**, câble USB-C (données), carte microSD (facultative).
- Une dizaine d'objets réels des quatre classes : boîte de céréales, canette, bouteille d'eau, pot en verre.
- Un fond neutre et un éclairage stable (le même qu'en TP2 pour pouvoir comparer).

Logiciel

1. **OpenMV IDE 4.x** et un firmware OpenMV récent ($\geq$ 4.5, qui fournit le module `ml`) (Tutorial 1).
2. Sur le PC (ou Google Colab) : Python 3.10+, `tensorflow` 2.15+, `numpy`, `pillow`.
3. Le jeu **TrashNet** (Thung & Yang, Stanford, 2016), dossiers `cardboard`, `metal`, `plastic`, `glass` : github.com/garythung/trashnet.

Astuce

Si vous n'avez pas le temps d'entraîner, utilisez le modèle fourni par l'enseignant (`materiau_int8.tflite`, `labels.txt`) et passez directement à l'étape 3. Alternative sans code : un projet Edge Impulse « Image classification », exporté avec **Deployment → OpenMV library**, fournit les mêmes deux fichiers.

3 Étapes détaillées du tutoriel

Étape 1 : Entraîner et quantifier le modèle (PC)

On reprend un MobileNetV2 $\alpha = 0{,}35$ pré-entraîné sur ImageNet (transfer learning) et l'on n'entraîne que la tête, puis on convertit en INT8 avec un jeu de calibration. Les entrées et sorties sont gardées en INT8 : c'est ce qu'attend la carte.

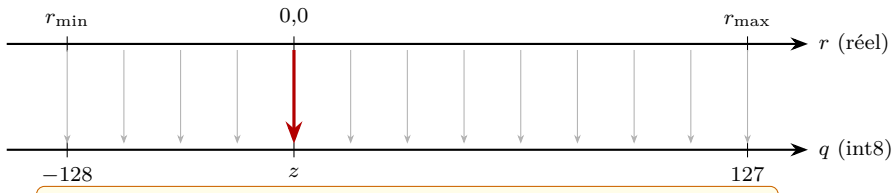
```
# entrainer_convertir.py - transfer learning + quantification INT8 (sur PC ou Colab)
import tensorflow as tf, numpy as np
TAILLE, CLASSES = 96, ["cardboard", "glass", "metal", "plastic"]
ds = tf.keras.utils.image_dataset_from_directory("trashnet", class_names=CLASSES,
        image_size=(TAILLE, TAILLE), batch_size=32, validation_split=0.2,
        subset="both", seed=42)
train, val = ds
aug = tf.keras.Sequential([tf.keras.layers.RandomFlip("horizontal"),
                           tf.keras.layers.RandomRotation(0.1),
                           tf.keras.layers.RandomBrightness(0.2)])
base = tf.keras.applications.MobileNetV2((TAILLE, TAILLE, 3), alpha=0.35,
                                         include_top=False, weights="imagenet")
base.trainable = False
inp = tf.keras.Input((TAILLE, TAILLE, 3))
x = tf.keras.layers.Rescaling(1 / 127.5, offset=-1)(aug(inp))   # [0,255] -> [-1,1]
x = tf.keras.layers.GlobalAveragePooling2D()(base(x, training=False))
out = tf.keras.layers.Dense(4, activation="softmax")(tf.keras.layers.Dropout(0.2)(x))
model = tf.keras.Model(inp, out)
model.compile("adam", "sparse_categorical_crossentropy", ["accuracy"])
model.fit(train, validation_data=val, epochs=12)
model.summary()                                    # notez le nombre de paramètres
```

```
# --- float32
open("materiau_f32.tflite", "wb").write(tf.lite.TFLiteConverter.from_keras_model(
    model).convert())
# --- INT8 complet (PTQ)
conv = tf.lite.TFLiteConverter.from_keras_model(model)
conv.optimizations = [tf.lite.Optimize.DEFAULT]
def calib():                                        # 200 images de calibration
    for x, _ in train.unbatch().take(200):
        yield [tf.expand_dims(tf.cast(x, tf.float32), 0)]
conv.representative_dataset = calib
conv.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
conv.inference_input_type = tf.int8
conv.inference_output_type = tf.int8
open("materiau_int8.tflite", "wb").write(conv.convert())
open("labels.txt", "w").write("\n".join(["carton", "verre", "metal", "plastique"]))
```

Question

Relevez la taille des deux fichiers `.tflite`. Le rapport est-il exactement 4 ? Pourquoi pas (indice : graphe, métadonnées, biais en INT32) ? Le nombre de paramètres affiché par `model.summary()` correspond-il aux 0,39 M du cours ?

Étape 2 : Inspecter le modèle quantifié (PC)



$q = \text{clamp}\left(\text{round}(r/s) + z, -128, 127\right)$ $\quad \hat{r} = s\,(q - z)$

$s = \dfrac{r_{\max} - r_{\min}}{255}$ (échelle), $\quad z$ = point zéro entier :
le réel 0 est **exactement** représentable (padding, ReLU)

Rappel du cours : chaque tenseur INT8 porte une échelle s et un point zéro z ; $\hat{r} = s\,(q - z)$.

```
# inspecter_tflite.py - entrée, sortie, quantification et validation sur PC
import tensorflow as tf, numpy as np
it = tf.lite.Interpreter("materiau_int8.tflite"); it.allocate_tensors()
e, s = it.get_input_details()[0], it.get_output_details()[0]
print("entrée :", e["shape"], e["dtype"], "s, z =", e["quantization"])
print("sortie :", s["shape"], s["dtype"], "s, z =", s["quantization"])
print("nombre de tenseurs :", len(it.get_tensor_details()))
# exactitude INT8 sur la validation : r -> q à l'entrée, q -> r à la sortie
(se, ze), (ss, zs) = e["quantization"], s["quantization"]
ok = n = 0
for x, y in val.unbatch():
    q = np.clip(np.round(x.numpy() / se + ze), -128, 127).astype(np.int8)
    it.set_tensor(e["index"], q[None]); it.invoke()
    p = (it.get_tensor(s["index"])[0].astype(np.float32) - zs) * ss
    ok += int(np.argmax(p) == y.numpy()); n += 1
print("exactitude INT8 validation : %.1f %%" % (100 * ok / n))
```

Question

Que valent s et z à l'entrée ? Vérifiez qu'ils correspondent à une image codée de 0 à 255 (pourquoi $z = -128$?). Comparez l'exactitude INT8 à l'exactitude float32 donnée par Keras : quelle est la perte due à la quantification ?

Étape 3 : Charger le modèle sur la Portenta et mesurer la mémoire

Copiez `materiau_int8.tflite`, `materiau_f32.tflite` et `labels.txt` à la racine du disque USB de la carte (ou de la microSD), puis éjectez proprement le disque et réinitialisez la carte.

```
# ml_charger.py - chargement et budget mémoire
import ml, gc, os, time
gc.collect(); libre0 = gc.mem_free()
for nom in ["materiau_int8.tflite", "materiau_f32.tflite"]:
    taille = os.stat(nom)[6]
    gc.collect()
    try:
        t0 = time.ticks_ms()
        # dans le framebuffer si le tas ne suffit pas (marge de 64 Ko)
        dans_fb = taille > (gc.mem_free() - 64 * 1024)
        m = ml.Model(nom, load_to_fb=dans_fb)
        dt = time.ticks_diff(time.ticks_ms(), t0)
        print(nom, ": %d Ko, chargé en %d ms, fb=%s" % (taille // 1024, dt, dans_fb))
        print("   entrée", m.input_shape, m.input_dtype, " sortie", m.output_shape)
        print("   tas consommé : %d Ko" % ((libre0 - gc.mem_free()) // 1024))
        del m
    except Exception as ex:
        print(nom, ": ÉCHEC ->", ex)
    gc.collect()
```

— Si votre firmware est antérieur à 4.5, le module s'appelle `tf` : `net = tf.load(nom, load_to_fb=True)` et `net.classify(img)`.

— `load_to_fb=True` place le modèle dans la mémoire du framebuffer plutôt que dans le tas MicroPython, plus petit.

Question

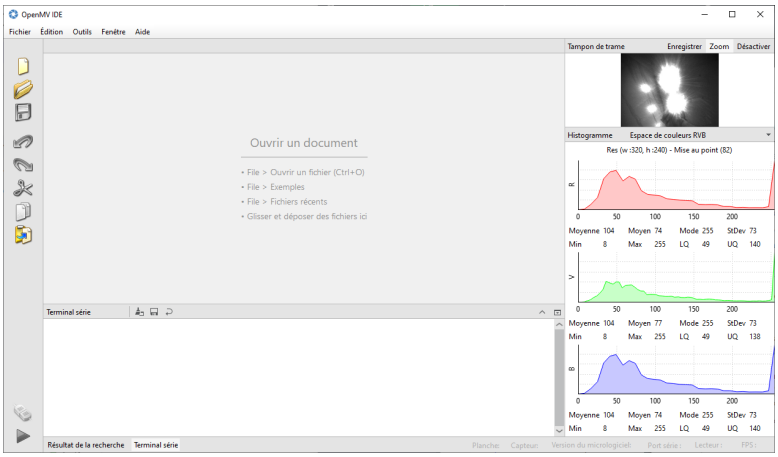
Le modèle float32 se charge-t-il ? Si oui, avec quelle marge ; sinon, quel message obtenez-vous ? Reliez la mémoire consommée à la taille du fichier : où se trouve le pic d'activations dans ce bilan ?

Étape 4 : Classer le flux de la caméra

```
# ml_classer.py - classification temps réel et mesure de latence
import sensor, time, ml, gc, os
sensor.reset()
sensor.set_pixformat(sensor.RGB565)          # GRAYSCALE avec le HM01B0 monochrome
sensor.set_framesize(sensor.QVGA)            # 320x240
sensor.set_windowing((240, 240))             # zone carrée centrale
sensor.skip_frames(time=2000)
sensor.set_auto_whitebal(False)              # comme au TP2 : conditions figées

NOM = "materiau_int8.tflite"
m = ml.Model(NOM, load_to_fb=os.stat(NOM)[6] > gc.mem_free() - 64 * 1024)
labels = [l.rstrip("\n") for l in open("labels.txt")]
```

```
horloge = time.clock(); lat = []
while True:
    horloge.tick()
    img = sensor.snapshot()
    t0 = time.ticks_us()
    p = m.predict([img])[0].flatten().tolist()    # redimensionnement 96x96
    automatique
    lat.append(time.ticks_diff(time.ticks_us(), t0) / 1000)
    k = p.index(max(p))
    img.draw_string(4, 4, "%s %.2f" % (labels[k], p[k]), color=(255, 0, 0), scale=2)
    if len(lat) == 50:
        lat.sort()
        print("latence médiane %.1f ms, P95 %.1f ms, %.1f FPS" % (lat[25], lat[47],
    horloge.fps()))
        lat = []
```



Le tampon d'image d'OpenMV IDE affiche la classe et la confiance ; le terminal série affiche latence et FPS.

Question

Comparez la latence médiane et le P95. D'où vient la différence ? La latence est-elle compatible avec le budget de 100 ms ? Estimez le débit en MACs par cycle : $10{,}7 \cdot 10^6$ MACs divisés par (latence $\times$ 480 MHz).

Étape 5 : Mesurer l'effet du prétraitement

Le modèle a été entraîné sur des photos TrashNet (fond blanc, 512×384 recadrées en 96×96). Modifiez une seule chose à la fois et notez la confiance moyenne sur 20 images du même objet.

Configuration	Classe majoritaire	Confiance moyenne
QVGA, fenêtre 240×240, fond blanc		
QVGA, image entière 320×240 (déformée)		
Fond du tapis (sombre)		
Balance des blancs automatique		
Objet à moitié hors champ		

Tableau à compléter. Conclusion attendue : le décalage de domaine (TrashNet → station) justifie la collecte d'images de la station au TP5.

Étape 6 : Vérifier la cohérence PC / carte

La même image doit donner la même prédiction sur le PC et sur la carte. Enregistrez 20 images JPEG avec la carte, puis comparez.

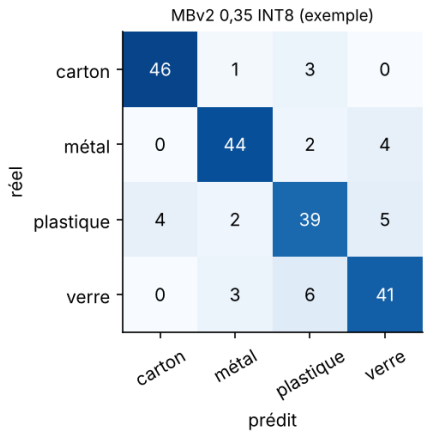
```
# ml_coherence.py - capture d'images de test et prédictions de la carte
import sensor, ml, os, gc
sensor.reset(); sensor.set_pixformat(sensor.RGB565); sensor.set_framesize(sensor.QVGA
    )
sensor.set_windowing((240, 240)); sensor.skip_frames(time=2000)
m = ml.Model("materiau_int8.tflite", load_to_fb=True)
with open("carte_predictions.csv", "w") as f:
    for i in range(20):
        sensor.skip_frames(time=1500)            # changez d'objet entre deux
    captures
        img = sensor.snapshot()
        img.save("test_%02d.jpg" % i, quality=95)
        p = m.predict([img])[0].flatten().tolist()
        f.write("test_%02d.jpg,%s\n" % (i, ",".join("%.3f" % v for v in p)))
        print(i, ["%.2f" % v for v in p])
```

Sur le PC, appliquez `inspecter_tflite.py` à ces 20 JPEG (redimensionnés en 96×96) et comparez les vecteurs de probabilités.

Question

Combien d'images ont la même classe prédite sur le PC et sur la carte ? Si des écarts apparaissent, quelles différences de prétraitement peuvent les expliquer (méthode de redimensionnement, conversion RGB565 vers RGB888, compression JPEG) ?

Étape 7 : Décider avec un seuil de confiance



Rappel du cours : les confusions verre / plastique sont les plus coûteuses ; un objet incertain est dévié vers le tri manuel.

```
# ml_decision.py - décision de tri avec seuil et vote sur 3 images
import sensor, ml, time, json
from pyb import LED
sensor.reset(); sensor.set_pixformat(sensor.RGB565); sensor.set_framesize(sensor.QVGA
    )
sensor.set_windowing((240, 240)); sensor.skip_frames(time=2000)
m = ml.Model("materiau_int8.tflite", load_to_fb=True)
labels = [l.rstrip("\n") for l in open("labels.txt")]
SEUIL = 0.70                                    # en dessous : tri manuel
vert, rouge = LED(2), LED(1)

def decider(n=3):
    votes = {}
    for _ in range(n):
        p = m.predict([sensor.snapshot()])[0].flatten().tolist()
        k = p.index(max(p)); votes[k] = votes.get(k, []) + [p[k]]
    k = max(votes, key=lambda c: len(votes[c]))
    conf = sum(votes[k]) / n                    # confiance moyenne sur les n images
    return (labels[k] if conf >= SEUIL else "manuel"), conf

while True:
    t0 = time.ticks_ms()
    classe, conf = decider()
    msg = json.dumps({"src": "vision", "mat": classe, "c": round(conf, 2),
                      "ms": time.ticks_diff(time.ticks_ms(), t0)})
    print(msg)
    (vert.on(), rouge.off()) if classe != "manuel" else (rouge.on(), vert.off())
    time.sleep_ms(500)
```

— Pour publier vers le smartphone, réutilisez la caractéristique DECISION du Tutorial 3 : `decision.write(msg, send_update=True)`.

— Faites varier `SEUIL` (0,5 ; 0,7 ; 0,9) : comment évoluent le taux de « manuel » et le taux d'erreur ?

Étape 8 : Relier les mesures au cours

Mesure	Valeur	Conditions
Taille fichier float32 / INT8 (Ko)		
Exactitude validation float32 / INT8 (PC, %)		TrashNet, 4 classes
Mémoire consommée au chargement INT8 (Ko)		`load_to_fb` = ?
Latence médiane / P95 (ms)		96×96, INT8
FPS de la boucle complète		capture + inférence + affichage
Accord PC / carte (sur 20 images)		
Taux de « manuel » à seuil 0,7		20 objets de la station

Tableau à compléter : il fournit la première ligne du tableau comparatif du TP5.

1. Le classifieur à règles J2 prenait quelques millisecondes : combien de fois plus lent est le CNN ? Le gain en exactitude le justifie-t-il ?
2. Quelle part du temps CPU reste-t-il pour le modèle audio du Tutorial 4 si l'on classe un objet par seconde ?
3. Le modèle float32 n'est-il qu'une question de mémoire ? Si vous avez pu le charger, mesurez sa latence et calculez l'accélération apportée par l'INT8.

Astuce

Pour un programme autonome, copiez le modèle et les étiquettes sur la carte, puis enregistrez le script avec **Tools → Save open script to OpenMV Cam (as main.py)**.

4 Pour aller plus loin

— Remplacez le modèle par un modèle **FOMO** exporté d'Edge Impulse et affichez les centroïdes de plusieurs objets (exemple OpenMV *Machine Learning → TensorFlow → FOMO*).

— Quantifiez avec `tf.lite.Optimize.DEFAULT` sans jeu de calibration (*dynamic range*) : le fichier rétrécit-il ? Le modèle s'exécute-t-il en entier sur la carte ?

— Entraînez en 128×128 et en 64×64 : tracez exactitude et latence, premiers points de votre front de Pareto.

— Documentation : module `ml` sur docs.openmv.io ; guide « Post-training quantization » sur tensorflow.org ; cours MIT 6.5940 (S. Han), chapitre Quantization.